Code:

```
# DAA Assignment 1: Fibonacci numbers - non-recursive and recursive
# Also shows time and space behaviour of both methods.

import time

call_count = 0   # counts how many times the recursive function is called


def fib_iterative(n):
    """Non-recursive (space optimised): keeps only the last two numbers.
    Time = O(n), Space = O(1)"""
    if n <= 1:
        return n
    a, b = 0, 1
    for _ in range(2, n + 1):
        a, b = b, a + b
    return b


def fib_series_iterative(n):
    """Non-recursive using an array F[] (dynamic programming style).
    Time = O(n), Space = O(n)"""
    f = [0, 1]
    for i in range(2, n):
        f.append(f[i - 1] + f[i - 2])
    return f[:n]


def fib_recursive(n):
    """Recursive: F(n) = F(n-1) + F(n-2).
    Time = O(2^n), Space = O(n) (depth of the recursion stack)"""
    global call_count
    call_count += 1
    if n <= 1:
        return n
    return fib_recursive(n - 1) + fib_recursive(n - 2)


# ---------------- Main program ----------------
n = 10

print("Fibonacci series of", n, "terms")
print("Non-recursive :", fib_series_iterative(n))
print("Recursive     :", [fib_recursive(i) for i in range(n)])

print("\nF(n) for n = 0..10 (non-recursive vs recursive)")
print("n   iterative   recursive")
for i in range(11):
    print(f"{i:<3} {fib_iterative(i):<11} {fib_recursive(i)}")

print("\nTime and number of calls for larger values of n")
print("n    value         iter time(s)   rec time(s)   rec calls")
for n in [10, 20, 25, 30]:
    t1 = time.perf_counter()
    v1 = fib_iterative(n)
    t1 = time.perf_counter() - t1

    call_count = 0
    t2 = time.perf_counter()
    v2 = fib_recursive(n)
    t2 = time.perf_counter() - t2

    assert v1 == v2          # both methods must give the same answer
    print(f"{n:<4} {v1:<13} {t1:<14.6f} {t2:<13.6f} {call_count}")

print("\nComplexity summary")
print("Method                 Time      Space")
print("Recursive              O(2^n)    O(n)")
print("Iterative with array   O(n)      O(n)")
print("Iterative (2 vars)     O(n)      O(1)")
```

Output:

```
Fibonacci series of 10 terms
Non-recursive : [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
Recursive     : [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

F(n) for n = 0..10 (non-recursive vs recursive)
```

```
n   iterative   recursive
0   0           0
1   1           1
2   1           1
3   2           2
4   3           3
5   5           5
6   8           8
7   13          13
8   21          21
9   34          34
10  55          55

Time and number of calls for larger values of n
n    value         iter time(s)   rec time(s)   rec calls
10   55            0.000001       0.000012      177
20   6765          0.000001       0.001158      21891
25   75025         0.000001       0.012341      242785
30   832040        0.000001       0.140397      2692537

Complexity summary
Method                Time      Space
Recursive             O(2^n)    O(n)
Iterative with array  O(n)      O(n)
Iterative (2 vars)    O(n)      O(1)
```

Code:

```
# DAA Assignment 2: Huffman Encoding using a greedy strategy

import heapq
from collections import Counter


class Node:
    def __init__(self, freq, char=None, left=None, right=None):
        self.freq = freq      # frequency of the character (or sum for internal node)
        self.char = char      # character (None for internal nodes)
        self.left = left
        self.right = right


def build_huffman_tree(freq):
    """Greedy step: always merge the two nodes with the smallest frequency."""
    heap = []
    for i, (ch, f) in enumerate(freq.items()):
        # i is a tie-breaker so that two nodes with equal frequency can be compared
        heapq.heappush(heap, (f, i, Node(f, ch)))
    counter = len(heap)

    while len(heap) > 1:
        f1, _, x = heapq.heappop(heap)      # smallest
        f2, _, y = heapq.heappop(heap)      # second smallest
        z = Node(f1 + f2, None, x, y)       # new internal node
        print(f"Merge {f1} + {f2} = {f1 + f2}")
        heapq.heappush(heap, (f1 + f2, counter, z))
        counter += 1
    return heap[0][2]


def generate_codes(node, code="", codes=None):
    """Left edge = 0, right edge = 1. Leaf nodes hold the characters."""
    if codes is None:
        codes = {}
    if node.char is not None:
        codes[node.char] = code or "0"      # "0" if there is only one symbol
        return codes
    generate_codes(node.left, code + "0", codes)
    generate_codes(node.right, code + "1", codes)
    return codes


def encode(text, codes):
    return "".join(codes[c] for c in text)


def decode(bits, root):
    result, node = "", root
    for b in bits:
        node = node.left if b == "0" else node.right
        if node.char is not None:
            result += node.char
            node = root
    return result


def show_table(freq, codes):
    print("Char  Freq  Code        Bits")
    for ch in sorted(codes, key=lambda c: (len(codes[c]), codes[c])):
        print(f"{ch!r:<5} {freq[ch]:<5} {codes[ch]:<11} {len(codes[ch])}")
    total = sum(freq.values())
    avg = sum(freq[c] * len(codes[c]) for c in freq) / total
    bits = sum(freq[c] * len(codes[c]) for c in freq)
    print(f"Average code length = {avg:.4f} bits/symbol")
    print(f"Total bits (Huffman) = {bits}   Total bits (8-bit ASCII) = {total * 8}")


# ---------------- Example 1: example from the practical ----------------
print("=== Example from the practical (a:5 b:9 c:12 d:13 e:16 f:45) ===")
freq = {"a": 5, "b": 9, "c": 12, "d": 13, "e": 16, "f": 45}
root = build_huffman_tree(freq)
codes = generate_codes(root)
show_table(freq, codes)

# ---------------- Example 2: viva question 'mississippi' ----------------
print("\n=== Viva question: message 'mississippi' ===")
```

```
text = "mississippi"
freq = Counter(text)
print("Frequencies:", dict(freq))
root = build_huffman_tree(freq)
codes = generate_codes(root)
show_table(freq, codes)
bits = encode(text, codes)
print("Encoded message :", bits)
print("Bits required   :", len(bits), "(fixed 8-bit ASCII would need", len(text) * 8, ")")
print("Decoded message :", decode(bits, root))

# ---------------- Example 3: assignment question 4 ----------------
print("\n=== Assignment question 4 text ===")
text = "ABCCDEBABFFBACBEBDFAAAABCDEEDCCBFEBFCAEFBACBEBDFAAAABC"
freq = Counter(text)
print("Text length:", len(text))
print("Frequencies:", dict(sorted(freq.items())))
root = build_huffman_tree(freq)
codes = generate_codes(root)
show_table(freq, codes)
bits = encode(text, codes)
print("Encoded length :", len(bits), "bits")
print("Decoded equals original text:", decode(bits, root) == text)
```

Output:

```
=== Example from the practical (a:5 b:9 c:12 d:13 e:16 f:45) ===
Merge 5 + 9 = 14
Merge 12 + 13 = 25
Merge 14 + 16 = 30
Merge 25 + 30 = 55
Merge 45 + 55 = 100
Char  Freq  Code        Bits
'f'   45    0           1
'c'   12    100         3
'd'   13    101         3
'e'   16    111         3
'a'   5     1100        4
'b'   9     1101        4
Average code length = 2.2400 bits/symbol
Total bits (Huffman) = 224   Total bits (8-bit ASCII) = 800

=== Viva question: message 'mississippi' ===
Frequencies: {'m': 1, 'i': 4, 's': 4, 'p': 2}
Merge 1 + 2 = 3
Merge 3 + 4 = 7
Merge 4 + 7 = 11
Char  Freq  Code        Bits
's'   4     0           1
'i'   4     11          2
'm'   1     100         3
'p'   2     101         3
Average code length = 1.9091 bits/symbol
Total bits (Huffman) = 21   Total bits (8-bit ASCII) = 88
Encoded message : 100110011001110110111
Bits required   : 21 (fixed 8-bit ASCII would need 88 )
Decoded message : mississippi

=== Assignment question 4 text ===
Text length: 54
Frequencies: {'A': 13, 'B': 13, 'C': 9, 'D': 5, 'E': 7, 'F': 7}
Merge 5 + 7 = 12
Merge 7 + 9 = 16
Merge 12 + 13 = 25
Merge 13 + 16 = 29
Merge 25 + 29 = 54
Char  Freq  Code        Bits
'A'   13    01          2
'B'   13    10          2
'D'   5     000         3
'E'   7     001         3
'F'   7     110         3
'C'   9     111         3
Average code length = 2.5185 bits/symbol
Total bits (Huffman) = 136   Total bits (8-bit ASCII) = 432
Encoded length : 136 bits
Decoded equals original text: True
```

Code:

```
# DAA Assignment 3: 0/1 Knapsack problem using Dynamic Programming
# T[i][j] = max( T[i-1][j] , value_i + T[i-1][j - weight_i] )


def knapsack(weights, values, capacity):
    n = len(weights)

    # Step 1: table T with (n+1) rows and (capacity+1) columns, filled with 0
    T = [[0] * (capacity + 1) for _ in range(n + 1)]

    # Step 2: fill the table row by row
    for i in range(1, n + 1):
        for j in range(1, capacity + 1):
            if weights[i - 1] <= j:
                T[i][j] = max(T[i - 1][j], values[i - 1] + T[i - 1][j - weights[i - 1]])
            else:
                T[i][j] = T[i - 1][j]          # item does not fit

    # Step 3: trace back to find which items were taken
    selected = []
    j = capacity
    for i in range(n, 0, -1):
        if T[i][j] != T[i - 1][j]:              # value changed -> item i was taken
            selected.append(i)
            j -= weights[i - 1]
    selected.reverse()
    return T, selected


def print_table(T, capacity):
    print("      " + "".join(f"j={j:<4}" for j in range(capacity + 1)))
    for i, row in enumerate(T):
        print(f"i={i}   " + "".join(f"{v:<6}" for v in row))


def solve(title, weights, values, capacity):
    print("=" * 60)
    print(title)
    print("Item   Weight   Value")
    for k in range(len(weights)):
        print(f"{k + 1:<6} {weights[k]:<8} {values[k]}")
    print("Knapsack capacity =", capacity, "kg\n")

    T, selected = knapsack(weights, values, capacity)
    print("DP table T:")
    print_table(T, capacity)

    print("\nMaximum value  =", T[-1][-1])
    print("Items selected =", selected)
    print("Total weight   =", sum(weights[i - 1] for i in selected))
    print("Total value    =", sum(values[i - 1] for i in selected))


# Problem solved in the practical
solve("Problem from the practical", [2, 3, 4, 5], [3, 4, 5, 6], 5)

# Assignment question 2: weights {3,4,6,5}, profits {2,3,1,4}, capacity 8 kg
solve("Assignment question 2", [3, 4, 6, 5], [2, 3, 1, 4], 8)

print("\nTime complexity  : O(n * W)  (table of (n+1) x (W+1) entries)")
print("Space complexity : O(n * W)  (O(W) if only the maximum value is needed)")
```

Output:

```
============================================================
Problem from the practical
Item   Weight   Value
1      2        3
2      3        4
3      4        5
4      5        6
Knapsack capacity = 5 kg

DP table T:
      j=0   j=1   j=2   j=3   j=4   j=5
i=0   0     0     0     0     0     0
i=1   0     0     3     3     3     3
i=2   0     0     3     4     4     7
```

```
i=3   0     0     3     4     5     7
i=4   0     0     3     4     5     7

Maximum value  = 7
Items selected = [1, 2]
Total weight   = 5
Total value    = 7
============================================================
Assignment question 2
Item   Weight   Value
1      3        2
2      4        3
3      6        1
4      5        4
Knapsack capacity = 8 kg

DP table T:
      j=0   j=1   j=2   j=3   j=4   j=5   j=6   j=7   j=8
i=0   0     0     0     0     0     0     0     0     0
i=1   0     0     0     2     2     2     2     2     2
i=2   0     0     0     2     3     3     3     5     5
i=3   0     0     0     2     3     3     3     5     5
i=4   0     0     0     2     3     4     4     5     6

Maximum value  = 6
Items selected = [1, 4]
Total weight   = 8
Total value    = 6

Time complexity  : O(n * W)  (table of (n+1) x (W+1) entries)
Space complexity : O(n * W)  (O(W) if only the maximum value is needed)
```

Code:

```
# DAA Assignment 4: N-Queens with the first Queen already placed,
# remaining Queens are placed using backtracking.
# (Rows and columns are numbered from 1 in the printed messages.)


def is_safe(board, row, col, n):
    """board[r] = column of the queen in row r (or -1 if empty).
    A position is safe if no earlier queen is in the same column or diagonal."""
    for r in range(n):
        c = board[r]
        if c == -1 or r == row:
            continue
        if c == col or abs(c - col) == abs(r - row):
            return False
    return True


def solve(board, row, n, first_row):
    """Place queens row by row; backtrack when no safe column is found."""
    if row == n:
        return True                     # all queens placed
    if row == first_row:
        return solve(board, row + 1, n, first_row)   # already has the first queen

    for col in range(n):
        if is_safe(board, row, col, n):
            board[row] = col            # place queen
            if solve(board, row + 1, n, first_row):
                return True
            board[row] = -1             # backtrack (remove queen)
    return False


def n_queens(n, first_row, first_col):
    board = [-1] * n
    board[first_row] = first_col        # place the first queen
    if solve(board, 0, n, first_row):
        return board
    return None


def print_board(board):
    n = len(board)
    for r in range(n):
        print(" ".join("Q" if board[r] == c else "." for c in range(n)))


def is_valid_solution(board):
    """Independent check: no two queens share a column or a diagonal."""
    n = len(board)
    for r1 in range(n):
        for r2 in range(r1 + 1, n):
            if board[r1] == board[r2] or abs(board[r1] - board[r2]) == r2 - r1:
                return False
    return True


def run(n, first_row, first_col):
    print("-" * 40)
    print(f"N = {n}, first Queen at (row {first_row + 1}, column {first_col + 1})")
    board = n_queens(n, first_row, first_col)
    if board is None:
        print("No solution exists with the first Queen at this position.")
        return
    print_board(board)
    for r in range(n):
        print(f"Queen {r + 1} -> Row {r + 1}, Column {board[r] + 1}")
    print("Valid solution (no two queens attack each other):", is_valid_solution(board))


run(4, 0, 1)      # 4-Queens, first queen at row 1, column 2
run(4, 0, 0)      # 4-Queens, first queen at row 1, column 1 (no solution exists)
run(6, 0, 2)      # 6-Queens
run(8, 0, 0)      # 8-Queens, first queen in the corner
run(8, 3, 3)      # 8-Queens, first queen in the middle

print("-" * 40)
print("Time complexity  : O(N!) in the worst case (backtracking prunes most branches)")
```

```
print("Space complexity : O(N)  (one column number per row + recursion depth)")
```

Output:

```
----------------------------------------
N = 4, first Queen at (row 1, column 2)
. Q . .
. . . Q
Q . . .
. . Q .
Queen 1 -> Row 1, Column 2
Queen 2 -> Row 2, Column 4
Queen 3 -> Row 3, Column 1
Queen 4 -> Row 4, Column 3
Valid solution (no two queens attack each other): True
----------------------------------------
N = 4, first Queen at (row 1, column 1)
No solution exists with the first Queen at this position.
----------------------------------------
N = 6, first Queen at (row 1, column 3)
. . Q . . .
. . . . . Q
. Q . . . .
. . . . Q .
Q . . . . .
. . . Q . .
Queen 1 -> Row 1, Column 3
Queen 2 -> Row 2, Column 6
Queen 3 -> Row 3, Column 2
Queen 4 -> Row 4, Column 5
Queen 5 -> Row 5, Column 1
Queen 6 -> Row 6, Column 4
Valid solution (no two queens attack each other): True
----------------------------------------
N = 8, first Queen at (row 1, column 1)
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
Queen 1 -> Row 1, Column 1
Queen 2 -> Row 2, Column 5
Queen 3 -> Row 3, Column 8
Queen 4 -> Row 4, Column 6
Queen 5 -> Row 5, Column 3
Queen 6 -> Row 6, Column 7
Queen 7 -> Row 7, Column 2
Queen 8 -> Row 8, Column 4
Valid solution (no two queens attack each other): True
----------------------------------------
N = 8, first Queen at (row 4, column 4)
. Q . . . . . .
. . . . Q . . .
. . . . . . Q .
. . . Q . . . .
Q . . . . . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
Queen 1 -> Row 1, Column 2
Queen 2 -> Row 2, Column 5
Queen 3 -> Row 3, Column 7
Queen 4 -> Row 4, Column 4
Queen 5 -> Row 5, Column 1
Queen 6 -> Row 6, Column 8
Queen 7 -> Row 7, Column 6
Queen 8 -> Row 8, Column 3
Valid solution (no two queens attack each other): True
----------------------------------------
Time complexity  : O(N!) in the worst case (backtracking prunes most branches)
Space complexity : O(N)  (one column number per row + recursion depth)
```

Code:

```
# DAA Assignment 5: Analysis of Quick Sort - deterministic and randomized variant
# Deterministic  : pivot is always the last element
# Randomized     : pivot is chosen randomly from the sub-array

import math
import random
import sys
import time

sys.setrecursionlimit(10000)    # sorted input makes deterministic quick sort go deep

comparisons = 0                 # number of element comparisons made


def partition(arr, low, high):
    """Lomuto partition using arr[high] as the pivot."""
    global comparisons
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        comparisons += 1
        if arr[j] <= pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    arr[i + 1], arr[high] = arr[high], arr[i + 1]
    return i + 1


def quick_sort(arr, low, high):
    """Deterministic quick sort (pivot = last element)."""
    if low < high:
        p = partition(arr, low, high)
        quick_sort(arr, low, p - 1)
        quick_sort(arr, p + 1, high)


def randomized_partition(arr, low, high):
    r = random.randint(low, high)           # choose a random pivot
    arr[r], arr[high] = arr[high], arr[r]   # move it to the end
    return partition(arr, low, high)


def randomized_quick_sort(arr, low, high):
    """Randomized quick sort (pivot = random element)."""
    if low < high:
        p = randomized_partition(arr, low, high)
        randomized_quick_sort(arr, low, p - 1)
        randomized_quick_sort(arr, p + 1, high)


def run(sort_function, data):
    """Sort a copy of data, return (sorted list, comparisons, time in seconds)."""
    global comparisons
    arr = data[:]
    comparisons = 0
    start = time.perf_counter()
    sort_function(arr, 0, len(arr) - 1)
    elapsed = time.perf_counter() - start
    return arr, comparisons, elapsed


random.seed(42)

# ---------- Small example ----------
sample = [10, 7, 8, 9, 1, 5, 3, 12, 6]
print("Original array             :", sample)
print("Deterministic quick sort   :", run(quick_sort, sample)[0])
print("Randomized quick sort      :", run(randomized_quick_sort, sample)[0])

# ---------- Analysis on different inputs ----------
n = 1000
inputs = {
    "Random order": random.sample(range(n * 10), n),
    "Already sorted": list(range(n)),
    "Reverse sorted": list(range(n, 0, -1)),
    "All equal": [7] * n,
}
print(f"\nAnalysis for n = {n}")
```

```
print(f"{'Input':<16}{'Variant':<15}{'Comparisons':<13}{'Time (s)':<11}Sorted?")
for name, data in inputs.items():
    for label, func in [("Deterministic", quick_sort), ("Randomized", randomized_quick_sort)]:
        result, comp, t = run(func, data)
        print(f"{name:<16}{label:<15}{comp:<13}{t:<11.5f}{result == sorted(data)}")

# ---------- Growth of comparisons with n ----------
print("\nComparisons on already sorted input (worst case for deterministic)")
print(f"{'n':<8}{'Deterministic':<16}{'Randomized':<14}n*log2(n)")
for n in [100, 200, 400, 800]:
    data = list(range(n))
    _, c1, _ = run(quick_sort, data)
    _, c2, _ = run(randomized_quick_sort, data)
    print(f"{n:<8}{c1:<16}{c2:<14}{round(n * math.log2(n))}")

print("\nComplexity")
print("Deterministic  : best/average O(n log n), worst O(n^2) (sorted input, last-element pivot)")
print("Randomized     : expected O(n log n) for every input, worst case O(n^2) is very unlikely")
print("Note           : with all-equal keys even the random pivot gives O(n^2) (Lomuto partition)")
print("Space          : O(log n) average stack, O(n) worst case. Quick sort is not stable.")
```

Output:

```
Original array              : [10, 7, 8, 9, 1, 5, 3, 12, 6]
Deterministic quick sort    : [1, 3, 5, 6, 7, 8, 9, 10, 12]
Randomized quick sort       : [1, 3, 5, 6, 7, 8, 9, 10, 12]

Analysis for n = 1000
Input           Variant        Comparisons  Time (s)   Sorted?
Random order    Deterministic  10652        0.00072    True
Random order    Randomized     10309        0.00092    True
Already sorted  Deterministic  499500       0.03069    True
Already sorted  Randomized     11059        0.00092    True
Reverse sorted  Deterministic  499500       0.02485    True
Reverse sorted  Randomized     10558        0.00093    True
All equal       Deterministic  499500       0.03087    True
All equal       Randomized     499500       0.03109    True

Comparisons on already sorted input (worst case for deterministic)
n       Deterministic   Randomized    n*log2(n)
100     4950            618           664
200     19900           1487          1529
400     79800           3676          3458
800     319600          8955          7715

Complexity
Deterministic  : best/average O(n log n), worst O(n^2) (sorted input, last-element pivot)
Randomized     : expected O(n log n) for every input, worst case O(n^2) is very unlikely
Note           : with all-equal keys even the random pivot gives O(n^2) (Lomuto partition)
Space          : O(log n) average stack, O(n) worst case. Quick sort is not stable.
```

Code:

```
# ML Assignment 8: Predict the price of an Uber ride
# 1. Pre-process the dataset   2. Linear Regression and Random Forest
# 3. Evaluate with R2, RMSE etc. and compare

import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")                       # save plots to files (no window needed)
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error

pd.set_option("display.width", 110)          # show all columns in the printed output
pd.set_option("display.max_columns", 20)

BASE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(BASE, "outputs")
os.makedirs(OUT, exist_ok=True)

# ---------------- 1. Read and pre-process the dataset ----------------
df = pd.read_csv(os.path.join(BASE, "datasets", "uber.csv"))
print("Shape of raw data:", df.shape)
print(df.head())

print("\nColumns with missing values:")
print(df.isnull().sum()[df.isnull().sum() > 0])

# drop columns that are not useful, and rows with missing values
df = df.drop(columns=["Unnamed: 0", "key"])
df = df.dropna()
print("Shape after dropping useless columns and missing values:", df.shape)

# convert pickup_datetime to date/time features
df["pickup_datetime"] = pd.to_datetime(df["pickup_datetime"])
df["year"] = df["pickup_datetime"].dt.year
df["month"] = df["pickup_datetime"].dt.month
df["day"] = df["pickup_datetime"].dt.day
df["weekday"] = df["pickup_datetime"].dt.weekday
df["hour"] = df["pickup_datetime"].dt.hour
df = df.drop(columns=["pickup_datetime"])


# Haversine formula: shortest distance (km) between two points on a sphere
def haversine(lat1, lon1, lat2, lon2):
    r = 6371                                        # radius of the earth in km
    lat1, lon1, lat2, lon2 = map(np.radians, [lat1, lon1, lat2, lon2])
    a = np.sin((lat2 - lat1) / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin((lon2 - lon1) / 2) ** 2
    return 2 * r * np.arcsin(np.sqrt(a))


df["distance_km"] = haversine(df["pickup_latitude"], df["pickup_longitude"],
                              df["dropoff_latitude"], df["dropoff_longitude"])

# ---------------- Outliers ----------------
print("\nSummary before removing outliers:")
print(df[["fare_amount", "passenger_count", "distance_km"]].describe().round(2))

# box plots before removing outliers
fig, ax = plt.subplots(1, 3, figsize=(12, 4))
for a, col in zip(ax, ["fare_amount", "distance_km", "passenger_count"]):
    a.boxplot(df[col])
    a.set_title(col)
fig.suptitle("Box plots BEFORE removing outliers")
plt.tight_layout()
plt.savefig(os.path.join(OUT, "ML_UberFarePrediction_pract8_boxplot_before.png"), dpi=100)
plt.close()

before = len(df)
# 1) impossible values
df = df[(df["fare_amount"] > 0) & (df["passenger_count"].between(1, 6))]
df = df[(df["pickup_latitude"].between(-90, 90)) & (df["dropoff_latitude"].between(-90, 90))]
df = df[(df["pickup_longitude"].between(-180, 180)) & (df["dropoff_longitude"].between(-180, 180))]
df = df[df["distance_km"] > 0]
# 2) outliers using the box-plot (IQR) rule
def remove_outliers_iqr(data, col):
```

```
    q1, q3 = data[col].quantile(0.25), data[col].quantile(0.75)
    iqr = q3 - q1
    return data[(data[col] >= q1 - 1.5 * iqr) & (data[col] <= q3 + 1.5 * iqr)]


df = remove_outliers_iqr(df, "fare_amount")
df = remove_outliers_iqr(df, "distance_km")
print(f"\nRows removed as outliers/invalid: {before - len(df)}   Rows left: {len(df)}")

fig, ax = plt.subplots(1, 3, figsize=(12, 4))
for a, col in zip(ax, ["fare_amount", "distance_km", "passenger_count"]):
    a.boxplot(df[col])
    a.set_title(col)
fig.suptitle("Box plots AFTER removing outliers")
plt.tight_layout()
plt.savefig(os.path.join(OUT, "ML_UberFarePrediction_pract8_boxplot_after.png"), dpi=100)
plt.close()

# ---------------- Correlation ----------------
corr = df.corr()
print("\nCorrelation of every column with fare_amount:")
print(corr["fare_amount"].sort_values(ascending=False).round(3))

plt.figure(figsize=(9, 7))
plt.imshow(corr, cmap="coolwarm", vmin=-1, vmax=1)
plt.colorbar()
plt.xticks(range(len(corr)), corr.columns, rotation=90)
plt.yticks(range(len(corr)), corr.columns)
plt.title("Correlation matrix")
plt.tight_layout()
plt.savefig(os.path.join(OUT, "ML_UberFarePrediction_pract8_correlation.png"), dpi=100)
plt.close()

# ---------------- 2. Build the models ----------------
X = df.drop(columns=["fare_amount"])
y = df["fare_amount"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print("\nTrain size:", X_train.shape, " Test size:", X_test.shape)

lr = LinearRegression()
lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)

rf = RandomForestRegressor(n_estimators=50, max_depth=15, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)

# ---------------- 3. Evaluate and compare ----------------
rows = []
for name, pred in [("Linear Regression", lr_pred), ("Random Forest", rf_pred)]:
    mse = mean_squared_error(y_test, pred)
    rows.append([name, r2_score(y_test, pred), mse, np.sqrt(mse), mean_absolute_error(y_test, pred)])
result = pd.DataFrame(rows, columns=["Model", "R2", "MSE", "RMSE", "MAE"])
print("\nModel comparison on the test data:")
print(result.round(4).to_string(index=False))

best = result.loc[result["R2"].idxmax(), "Model"]
print("\nBest model (highest R2, lowest RMSE):", best)

print("\nRandom Forest feature importance:")
print(pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False).round(4))

# actual vs predicted plot
plt.figure(figsize=(10, 4))
for k, (name, pred) in enumerate([("Linear Regression", lr_pred), ("Random Forest", rf_pred)]):
    plt.subplot(1, 2, k + 1)
    plt.scatter(y_test[:1500], pred[:1500], s=6, alpha=0.5)
    plt.plot([0, 25], [0, 25], "r--")
    plt.xlabel("Actual fare")
    plt.ylabel("Predicted fare")
    plt.title(name)
plt.tight_layout()
plt.savefig(os.path.join(OUT, "ML_UberFarePrediction_pract8_actual_vs_predicted.png"), dpi=100)
plt.close()
```

Output:

```
Shape of raw data: (200000, 9)
   Unnamed: 0                            key  fare_amount          pickup_datetime  pickup_longitude  \
0    24238194  2015-05-07 19:52:06.000000300          7.5  2015-05-07 19:52:06 UTC        -73.999817
```

```
1    27835199  2009-07-17 20:04:56.000000200          7.7  2009-07-17 20:04:56 UTC        -73.994355
2    44984355  2009-08-24 21:45:00.000000610         12.9  2009-08-24 21:45:00 UTC        -74.005043
3    25894730  2009-06-26 08:22:21.000000100          5.3  2009-06-26 08:22:21 UTC        -73.976124
4    17610152  2014-08-28 17:47:00.000000188         16.0  2014-08-28 17:47:00 UTC        -73.925023

   pickup_latitude  dropoff_longitude  dropoff_latitude  passenger_count
0        40.738354         -73.999512         40.723217                1
1        40.728225         -73.994710         40.750325                1
2        40.740770         -73.962565         40.772647                1
3        40.790844         -73.965316         40.803349                3
4        40.744085         -73.973082         40.761247                5

Columns with missing values:
dropoff_longitude    1
dropoff_latitude     1
dtype: int64
Shape after dropping useless columns and missing values: (199999, 7)

Summary before removing outliers:
       fare_amount  passenger_count  distance_km
count    199999.00        199999.00    199999.00
mean         11.36             1.68        20.86
std           9.90             1.39       382.96
min         -52.00             0.00         0.00
25%           6.00             1.00         1.22
50%           8.50             1.00         2.12
75%          12.50             2.00         3.88
max         499.00           208.00     16409.24

Rows removed as outliers/invalid: 30980   Rows left: 169019

Correlation of every column with fare_amount:
fare_amount          1.000
distance_km          0.796
year                 0.173
month                0.033
passenger_count      0.017
weekday              0.013
hour                 0.007
day                  0.005
dropoff_longitude    0.003
pickup_longitude     0.002
pickup_latitude     -0.003
dropoff_latitude    -0.004
Name: fare_amount, dtype: float64

Train size: (135215, 11)  Test size: (33804, 11)

Model comparison on the test data:
            Model     R2    MSE   RMSE    MAE
Linear Regression 0.6730 4.5233 2.1268 1.5198
    Random Forest 0.7412 3.5804 1.8922 1.3179

Best model (highest R2, lowest RMSE): Random Forest

Random Forest feature importance:
distance_km          0.7606
year                 0.0549
dropoff_latitude     0.0367
dropoff_longitude    0.0292
hour                 0.0274
pickup_longitude     0.0254
pickup_latitude      0.0229
month                0.0140
day                  0.0123
weekday              0.0121
passenger_count      0.0044
dtype: float64
```

Figure: Boxplot Before

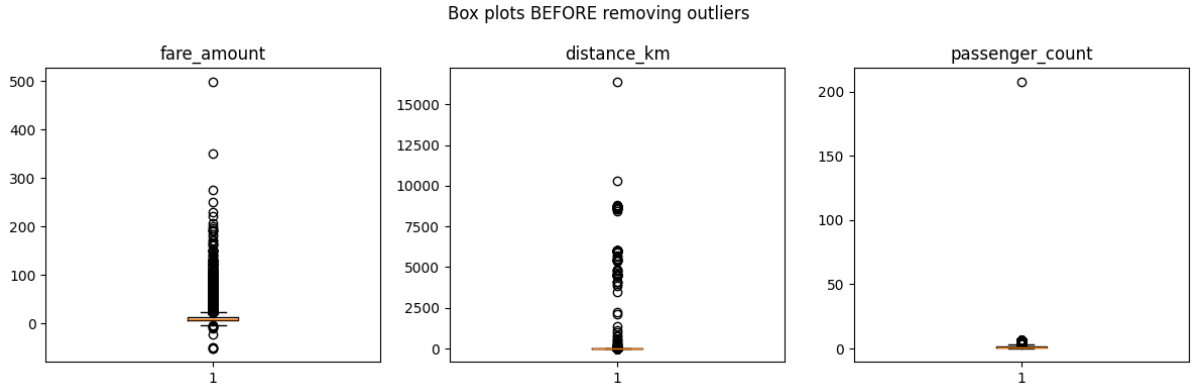


Figure: Boxplot After

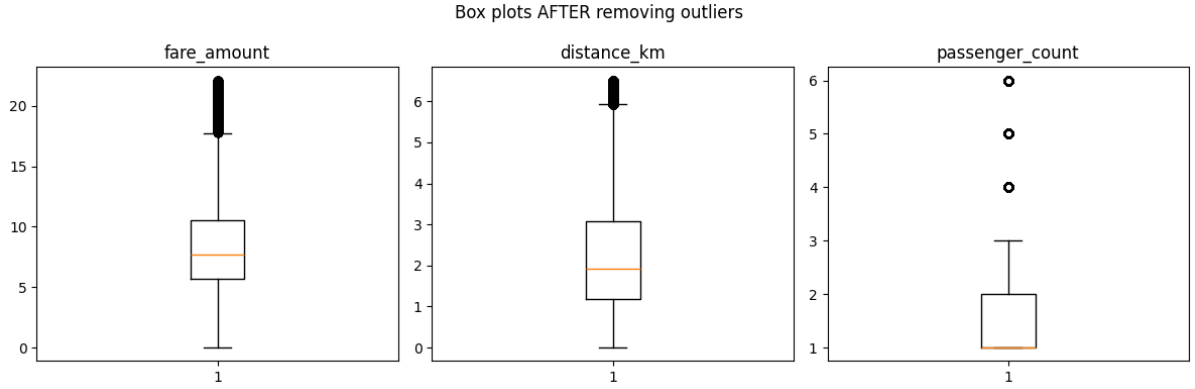


Figure: Correlation

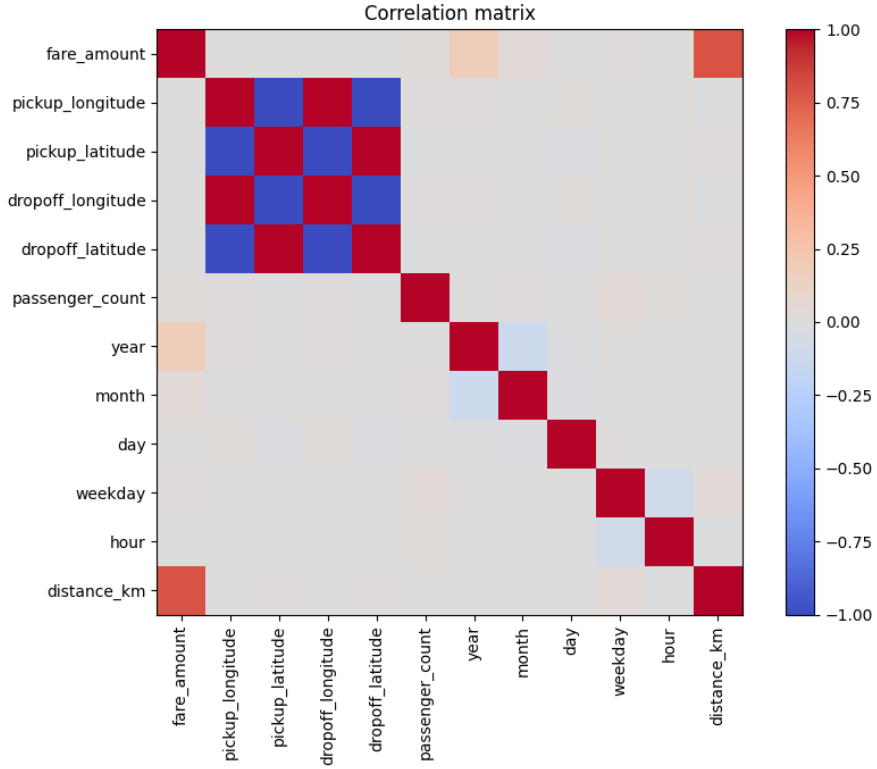
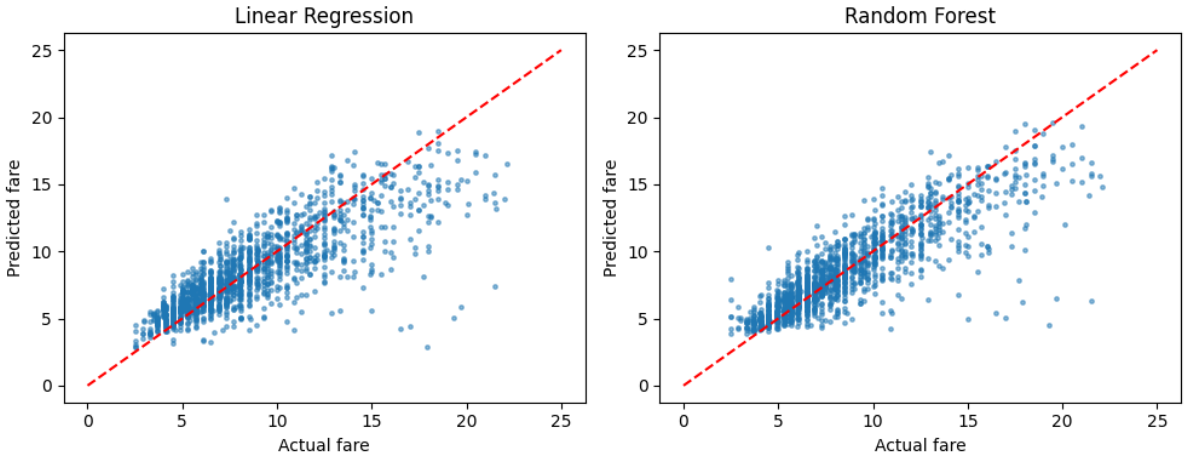


Figure: Actual Vs Predicted



Code:

```
# ML Assignment 9: Email spam detection (binary classification)
# Normal state = Not Spam (0), Abnormal state = Spam (1)
# Algorithms: K-Nearest Neighbors and Support Vector Machine

import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import normalize
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import (accuracy_score, confusion_matrix, precision_score,
                             recall_score, f1_score, classification_report)

BASE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(BASE, "outputs")
os.makedirs(OUT, exist_ok=True)

# ---------------- Data pre-processing ----------------
df = pd.read_csv(os.path.join(BASE, "datasets", "emails.csv"))
print("Shape of the dataset (emails, columns):", df.shape)
print("Missing values:", int(df.isnull().sum().sum()))
print("Not spam (0):", int((df["Prediction"] == 0).sum()), "  Spam (1):", int((df["Prediction"] == 1).sum()))

X = df.drop(columns=["Email No.", "Prediction"])      # 3000 word-count columns
y = df["Prediction"]                                   # label column
print("Features:", X.shape, " Target:", y.shape)

# word counts differ a lot between long and short emails, so scale every email
# (row) to unit length - this is the feature scaling step
X = pd.DataFrame(normalize(X), columns=X.columns)

# train / test split (80% / 20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print("Training emails:", len(X_train), "  Testing emails:", len(X_test))

# ---------------- Models ----------------
models = {
    "K-Nearest Neighbors (k=5)": KNeighborsClassifier(n_neighbors=5),
    "Support Vector Machine (RBF)": SVC(kernel="rbf", C=10),
}

results = []
plt.figure(figsize=(9, 4))
for k, (name, model) in enumerate(models.items()):
    model.fit(X_train, y_train)
    pred = model.predict(X_test)

    cm = confusion_matrix(y_test, pred)
    tn, fp, fn, tp = cm.ravel()
    acc = accuracy_score(y_test, pred)
    prec = precision_score(y_test, pred)
    rec = recall_score(y_test, pred)
    f1 = f1_score(y_test, pred)
    results.append([name, acc, 1 - acc, prec, rec, f1])

    print("\n" + "=" * 60)
    print(name)
    print("Confusion matrix [rows = actual, columns = predicted]:")
    print(cm)
    print(f"TN = {tn}, FP = {fp}, FN = {fn}, TP = {tp}")
    print(classification_report(y_test, pred, target_names=["Not Spam", "Spam"], digits=4))

    plt.subplot(1, 2, k + 1)
    plt.imshow(cm, cmap="Blues")
    for i in range(2):
        for j in range(2):
            plt.text(j, i, cm[i, j], ha="center", va="center", fontsize=14)
    plt.xticks([0, 1], ["Not Spam", "Spam"])
    plt.yticks([0, 1], ["Not Spam", "Spam"])
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title(name, fontsize=9)
plt.tight_layout()
```

```
plt.savefig(os.path.join(OUT, "ML_EmailSpamKNN_SVM_pract9_confusion_matrix.png"), dpi=100)
plt.close()

# ---------------- Compare the performance ----------------
table = pd.DataFrame(results, columns=["Model", "Accuracy", "Error rate", "Precision", "Recall", "F1-score"])
print("\n" + "=" * 60)
print("Performance comparison")
print(table.round(4).to_string(index=False))
best = table.loc[table["Accuracy"].idxmax(), "Model"]
print("\nBest model by accuracy:", best)
```

Output:

```
Shape of the dataset (emails, columns): (5171, 3002)
Missing values: 0
Not spam (0): 3672   Spam (1): 1499
Features: (5171, 3000)  Target: (5171,)
Training emails: 4136   Testing emails: 1035

============================================================
K-Nearest Neighbors (k=5)
Confusion matrix [rows = actual, columns = predicted]:
[[694  41]
 [ 32 268]]
TN = 694, FP = 41, FN = 32, TP = 268
              precision    recall  f1-score   support

    Not Spam     0.9559    0.9442    0.9500       735
        Spam     0.8673    0.8933    0.8801       300

    accuracy                         0.9295      1035
   macro avg     0.9116    0.9188    0.9151      1035
weighted avg     0.9302    0.9295    0.9298      1035


============================================================
Support Vector Machine (RBF)
Confusion matrix [rows = actual, columns = predicted]:
[[724  11]
 [  7 293]]
TN = 724, FP = 11, FN = 7, TP = 293
              precision    recall  f1-score   support

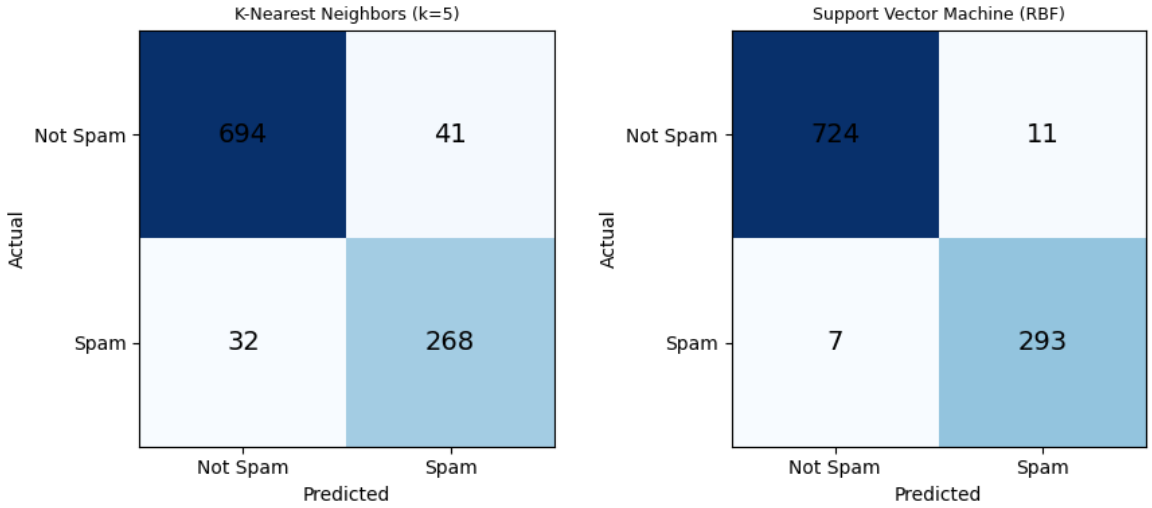
    Not Spam     0.9904    0.9850    0.9877       735
        Spam     0.9638    0.9767    0.9702       300

    accuracy                         0.9826      1035
   macro avg     0.9771    0.9809    0.9790      1035
weighted avg     0.9827    0.9826    0.9826      1035


============================================================
Performance comparison
                     Model  Accuracy  Error rate  Precision  Recall  F1-score
   K-Nearest Neighbors (k=5)    0.9295      0.0705     0.8673  0.8933    0.8801
Support Vector Machine (RBF)    0.9826      0.0174     0.9638  0.9767    0.9702

Best model by accuracy: Support Vector Machine (RBF)
```

Figure: Confusion Matrix



Code:

```
# ML Assignment 10: Neural network classifier - will a bank customer leave
# (Exited = 1) in the next 6 months or not?

import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"          # hide TensorFlow info messages
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report, recall_score, f1_score

pd.set_option("display.width", 110)
pd.set_option("display.max_columns", 20)
np.random.seed(1)
tf.random.set_seed(1)

BASE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(BASE, "outputs")
os.makedirs(OUT, exist_ok=True)

# ---------------- 1. Read the dataset ----------------
df = pd.read_csv(os.path.join(BASE, "datasets", "Churn_Modelling.csv"))
print("Shape:", df.shape)
print(df.head())
print("Missing values:", int(df.isnull().sum().sum()))
print("Customers who stayed (0):", int((df["Exited"] == 0).sum()), "  left (1):", int((df["Exited"] ==
    1).sum()))

# ---------------- 2. Feature set, target set, train/test split ----------------
df = df.drop(columns=["RowNumber", "CustomerId", "Surname"])    # not useful for prediction
df["Gender"] = df["Gender"].map({"Female": 0, "Male": 1})        # text -> number
df = pd.get_dummies(df, columns=["Geography"], dtype=int)        # one-hot encoding

X = df.drop(columns=["Exited"])          # features
y = df["Exited"]                         # target
print("\nFeatures used:", list(X.columns))

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1, stratify=y)
print("Train:", X_train.shape, " Test:", X_test.shape)

# ---------------- 3. Normalize the train and test data ----------------
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)      # learn mean/std from the training data only
X_test = scaler.transform(X_test)
print("Mean of first training feature after scaling: %.3f" % X_train[:, 0].mean())

n_features = X_train.shape[1]


def show_result(name, model):
    prob = model.predict(X_test, verbose=0).ravel()
    pred = (prob > 0.5).astype(int)
    print("\n" + "=" * 60)
    print(name)
    print("Accuracy score:", round(accuracy_score(y_test, pred), 4))
    print("Confusion matrix [rows = actual, columns = predicted]:")
    print(confusion_matrix(y_test, pred))
    print(classification_report(y_test, pred, target_names=["Stayed", "Left"], digits=4))
    return [name, accuracy_score(y_test, pred), recall_score(y_test, pred), f1_score(y_test, pred)]


# ---------------- 4a. Initialize and build the basic model ----------------
model1 = keras.Sequential([
    keras.layers.Input(shape=(n_features,)),
    keras.layers.Dense(6, activation="relu"),
    keras.layers.Dense(6, activation="relu"),
    keras.layers.Dense(1, activation="sigmoid"),
])
model1.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
print("\nBasic model:")
model1.summary()
hist1 = model1.fit(X_train, y_train, epochs=50, batch_size=32, verbose=0)
```

```
res1 = show_result("Basic model (6-6-1 neurons, 50 epochs)", model1)

# ---------------- 4b. Points of improvement ----------------
# The basic model finds only ~47% of the customers who really leave (low recall)
# because only 20% of customers leave. Improvements:
# 1) more neurons, 2) Dropout against over-fitting, 3) EarlyStopping on a validation
# split, 4) class weights so that the "Left" class counts more during training.
weights = {0: 1.0, 1: 2.0}
model2 = keras.Sequential([
    keras.layers.Input(shape=(n_features,)),
    keras.layers.Dense(32, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(16, activation="relu"),
    keras.layers.Dropout(0.2),
    keras.layers.Dense(1, activation="sigmoid"),
])
model2.compile(optimizer="adam", loss="binary_crossentropy", metrics=["accuracy"])
stop = keras.callbacks.EarlyStopping(monitor="val_loss", patience=10, restore_best_weights=True)
hist2 = model2.fit(X_train, y_train, epochs=100, batch_size=32, validation_split=0.2,
                   callbacks=[stop], class_weight=weights, verbose=0)
print("\nImproved model stopped after", len(hist2.history["loss"]), "epochs")
res2 = show_result("Improved model (32-16-1, Dropout, EarlyStopping, class weights)", model2)

# ---------------- Comparison ----------------
print("=" * 60)
print("Comparison (recall and F1 are for customers who LEFT)")
print(pd.DataFrame([res1, res2], columns=["Model", "Accuracy", "Recall",
    "F1-score"]).round(4).to_string(index=False))

# ---------------- Training curves ----------------
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(hist1.history["accuracy"], label="basic model")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("Basic model training accuracy")
plt.legend()
plt.subplot(1, 2, 2)
plt.plot(hist2.history["loss"], label="training loss")
plt.plot(hist2.history["val_loss"], label="validation loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Improved model loss")
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(OUT, "ML_BankChurnNeuralNetwork_pract10_training.png"), dpi=100)
plt.close()
```

Output:

```
Shape: (10000, 14)
   RowNumber  CustomerId   Surname  CreditScore Geography  Gender  Age  Tenure    Balance  NumOfProducts  \
0          1    15634602  Hargrave          619    France  Female   42       2       0.00              1
1          2    15647311      Hill          608     Spain  Female   41       1   83807.86              1
2          3    15619304      Onio          502    France  Female   42       8  159660.80              3
3          4    15701354      Boni          699    France  Female   39       1       0.00              2
4          5    15737888  Mitchell          850     Spain  Female   43       2  125510.82              1

   HasCrCard  IsActiveMember  EstimatedSalary  Exited
0          1               1        101348.88       1
1          0               1        112542.58       0
2          1               0        113931.57       1
3          0               0         93826.63       0
4          1               1         79084.10       0
Missing values: 0
Customers who stayed (0): 7963   left (1): 2037

Features used: ['CreditScore', 'Gender', 'Age', 'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard',
    'IsActiveMember', 'EstimatedSalary', 'Geography_France', 'Geography_Germany', 'Geography_Spain']
Train: (8000, 12)  Test: (2000, 12)
Mean of first training feature after scaling: 0.000

Basic model:
Model: "sequential"
■■■
```

```
┃ dense_2 (Dense)                 ┃ (None, 1)              ┃             7 ┃
 Total params: 127 (508.00 B)
 Trainable params: 127 (508.00 B)
 Non-trainable params: 0 (0.00 B)

============================================================
Basic model (6-6-1 neurons, 50 epochs)
Accuracy score: 0.86
Confusion matrix [rows = actual, columns = predicted]:
[[1525   68]
 [ 212  195]]
              precision    recall  f1-score   support

      Stayed     0.8780    0.9573    0.9159      1593
        Left     0.7414    0.4791    0.5821       407

    accuracy                         0.8600      2000
   macro avg     0.8097    0.7182    0.7490      2000
weighted avg     0.8502    0.8600    0.8480      2000


Improved model stopped after 47 epochs

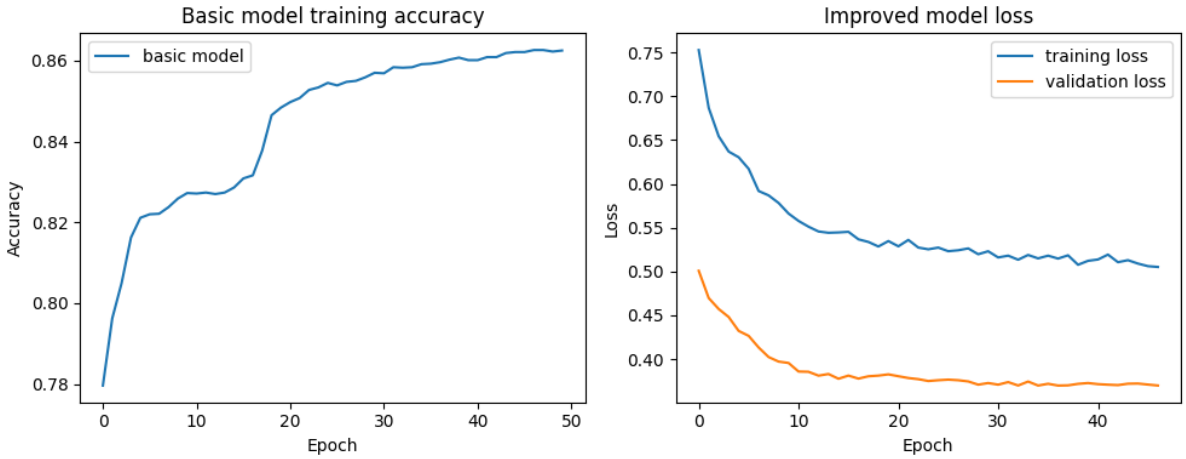
============================================================
Improved model (32-16-1, Dropout, EarlyStopping, class weights)
Accuracy score: 0.8535
Confusion matrix [rows = actual, columns = predicted]:
[[1468  125]
 [ 168  239]]
              precision    recall  f1-score   support

      Stayed     0.8973    0.9215    0.9093      1593
        Left     0.6566    0.5872    0.6200       407

    accuracy                         0.8535      2000
   macro avg     0.7770    0.7544    0.7646      2000
weighted avg     0.8483    0.8535    0.8504      2000

============================================================
Comparison (recall and F1 are for customers who LEFT)
                                                        Model  Accuracy  Recall  F1-score
                          Basic model (6-6-1 neurons, 50 epochs)    0.8600  0.4791    0.5821
Improved model (32-16-1, Dropout, EarlyStopping, class weights)    0.8535  0.5872    0.6200
```

Figure: Training



Code:

```
# ML Assignment 11: K-Nearest Neighbors on diabetes.csv
# Confusion matrix, accuracy, error rate, precision, recall (+ ROC/AUC,
# cross-validation, GridSearchCV and a Random Forest for comparison)

import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.metrics import (confusion_matrix, accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, roc_curve)

pd.set_option("display.width", 110)
pd.set_option("display.max_columns", 20)
BASE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(BASE, "outputs")
os.makedirs(OUT, exist_ok=True)

# ---------------- Read the data and pre-process ----------------
df = pd.read_csv(os.path.join(BASE, "datasets", "diabetes.csv"))
print("Shape (rows, columns):", df.shape)
print(df.head())
print("\nMissing values:", int(df.isnull().sum().sum()))

# In these columns a value of 0 is not possible, so 0 really means "missing"
cols = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
print("Zero values in", cols, ":", (df[cols] == 0).sum().to_dict())
df[cols] = df[cols].replace(0, np.nan)
df[cols] = df[cols].fillna(df[cols].median())     # fill with the median

print("\nCorrelation of each column with Outcome:")
print(df.corr()["Outcome"].sort_values(ascending=False).round(3))

# ---------------- Inputs (X) and target (y), train/test split ----------------
X = df.drop(columns=["Outcome"])
y = df["Outcome"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1, stratify=y)

# feature scaling: KNN is distance based, so all columns need the same scale
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)


def report(name, model):
    """Print confusion matrix, accuracy, error rate, precision and recall."""
    pred = model.predict(X_test)
    cm = confusion_matrix(y_test, pred)
    tn, fp, fn, tp = cm.ravel()
    acc = accuracy_score(y_test, pred)
    print("\n" + "=" * 55)
    print(name)
    print("Confusion matrix [rows = actual, columns = predicted]:")
    print(cm)
    print(f"TN = {tn}  FP = {fp}  FN = {fn}  TP = {tp}")
    print("Accuracy   :", round(acc, 4))
    print("Error rate :", round(1 - acc, 4))
    print("Precision  :", round(precision_score(y_test, pred), 4))
    print("Recall     :", round(recall_score(y_test, pred), 4))
    print("F1-score   :", round(f1_score(y_test, pred), 4))
    auc = roc_auc_score(y_test, model.predict_proba(X_test)[:, 1])
    print("ROC AUC    :", round(auc, 4))
    return auc


# ---------------- KNN with k = 3 (starting point) ----------------
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)
auc_knn3 = report("KNN (k = 3)", knn)
print("First 5 predictions:", knn.predict(X_test)[:5], " actual:", y_test.values[:5])
# ---------------- k-fold cross validation ----------------
```

```
# the pipeline scales the data inside every fold, so no test data leaks into training
cv_scores = cross_val_score(make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=3)), X, y, cv=5)
print("\n5-fold cross validation scores:", cv_scores.round(4))
print("Mean cross validation accuracy :", round(cv_scores.mean(), 4))

# ---------------- GridSearchCV: find the best k ----------------
grid = GridSearchCV(KNeighborsClassifier(), {"n_neighbors": np.arange(1, 25)}, cv=5)
grid.fit(X_train, y_train)
print("\nBest value of k  :", grid.best_params_["n_neighbors"])
print("Best CV accuracy :", round(grid.best_score_, 4))

best_knn = grid.best_estimator_
auc_best = report(f"KNN with best k = {grid.best_params_['n_neighbors']}", best_knn)

# ---------------- Random Forest for comparison ----------------
rf = RandomForestClassifier(n_estimators=100, random_state=1)
rf.fit(X_train, y_train)
auc_rf = report("Random Forest (100 trees)", rf)

# ---------------- ROC curves ----------------
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
for name, model in [("KNN k=3", knn), ("KNN best k", best_knn), ("Random Forest", rf)]:
    fpr, tpr, _ = roc_curve(y_test, model.predict_proba(X_test)[:, 1])
    plt.plot(fpr, tpr, label=name)
plt.plot([0, 1], [0, 1], "k--")
plt.xlabel("False positive rate")
plt.ylabel("True positive rate")
plt.title("ROC curves")
plt.legend()

plt.subplot(1, 2, 2)
plt.plot(grid.cv_results_["param_n_neighbors"].data.astype(int), grid.cv_results_["mean_test_score"],
    marker="o")
plt.xlabel("k (n_neighbors)")
plt.ylabel("CV accuracy")
plt.title("Choosing k with GridSearchCV")
plt.tight_layout()
plt.savefig(os.path.join(OUT, "ML_DiabetesKNN_pract11_roc_and_k.png"), dpi=100)
plt.close()
```

Output:

```
Shape (rows, columns): (768, 9)
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  DiabetesPedigreeFunction  Age  Outcome
0            6      148             72             35        0  33.6                     0.627   50        1
1            1       85             66             29        0  26.6                     0.351   31        0
2            8      183             64              0        0  23.3                     0.672   32        1
3            1       89             66             23       94  28.1                     0.167   21        0
4            0      137             40             35      168  43.1                     2.288   33        1

Missing values: 0
Zero values in ['Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI'] : {'Glucose': 5, 'BloodPressure':
     35, 'SkinThickness': 227, 'Insulin': 374, 'BMI': 11}

Correlation of each column with Outcome:
Outcome                    1.000
Glucose                    0.493
BMI                        0.312
Age                        0.238
Pregnancies                0.222
SkinThickness              0.215
Insulin                    0.204
DiabetesPedigreeFunction   0.174
BloodPressure              0.166
Name: Outcome, dtype: float64

======================================================
KNN (k = 3)
Confusion matrix [rows = actual, columns = predicted]:
[[86 14]
 [24 30]]
TN = 86  FP = 14  FN = 24  TP = 30
Accuracy   : 0.7532
Error rate : 0.2468
Precision  : 0.6818
Recall     : 0.5556
F1-score   : 0.6122
ROC AUC    : 0.7854
First 5 predictions: [0 0 0 1 1]  actual: [0 0 0 0 0]
```

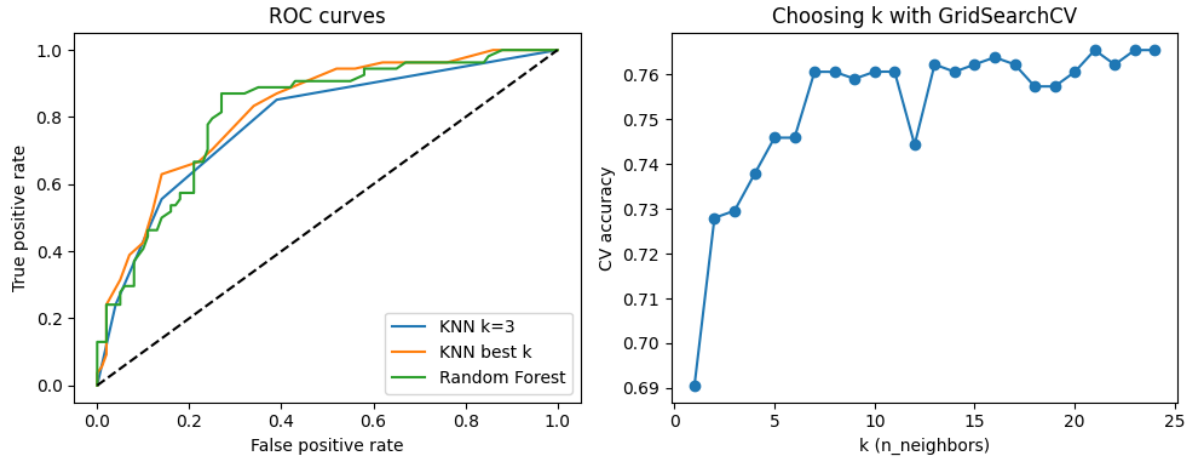
```
5-fold cross validation scores: [0.7532 0.7338 0.6883 0.7843 0.732 ]
Mean cross validation accuracy : 0.7383

Best value of k  : 24
Best CV accuracy : 0.7654

=======================================================
KNN with best k = 24
Confusion matrix [rows = actual, columns = predicted]:
[[90 10]
 [31 23]]
TN = 90  FP = 10  FN = 31  TP = 23
Accuracy   : 0.7338
Error rate : 0.2662
Precision  : 0.697
Recall     : 0.4259
F1-score   : 0.5287
ROC AUC    : 0.8198

=======================================================
Random Forest (100 trees)
Confusion matrix [rows = actual, columns = predicted]:
[[84 16]
 [26 28]]
TN = 84  FP = 16  FN = 26  TP = 28
Accuracy   : 0.7273
Error rate : 0.2727
Precision  : 0.6364
Recall     : 0.5185
F1-score   : 0.5714
ROC AUC    : 0.813
```

Figure: Roc And K



Code:

```
# ML Assignment 12: K-Means and Hierarchical clustering on customer data
# (Mall customers). Number of clusters is found with the elbow method.

import os
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import dendrogram, linkage

pd.set_option("display.width", 110)
pd.set_option("display.max_columns", 20)
BASE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(BASE, "outputs")
os.makedirs(OUT, exist_ok=True)

# ---------------- Read and explore the data ----------------
data = pd.read_csv(os.path.join(BASE, "datasets", "Mall_Customers.csv"))
data = data.rename(columns={"Genre": "Gender", "Annual_Income_(k$)": "Annual Income (k$)",
                            "Spending_Score": "Spending Score (1-100)"})
print("Shape:", data.shape)
print(data.head())
print("\nMissing values:", int(data.isnull().sum().sum()))
print("\nStatistics:")
print(data.describe().round(2))
print("\nCorrelation:")
print(data[["Age", "Annual Income (k$)", "Spending Score (1-100)"]].corr().round(3))

# features used for clustering: annual income and spending score
X = data[["Annual Income (k$)", "Spending Score (1-100)"]].values

# ---------------- Elbow method ----------------
wcss = []                                   # within cluster sum of squares
for k in range(1, 11):
    km = KMeans(n_clusters=k, init="k-means++", n_init=10, random_state=42)
    km.fit(X)
    wcss.append(km.inertia_)
print("\nWCSS for k = 1 to 10:")
for k, w in enumerate(wcss, start=1):
    print(f"k = {k:<2} WCSS = {w:,.1f}")

plt.figure(figsize=(8, 4))
plt.plot(range(1, 11), wcss, marker="o", color="red", linewidth=2)
plt.xticks(range(1, 11))
plt.xlabel("Number of clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow method")
plt.grid(True)
plt.savefig(os.path.join(OUT, "ML_KMeansClustering_pract12_elbow.png"), dpi=100)
plt.close()

# choose the elbow automatically: k after which the drop in WCSS becomes small
drops = [wcss[i] - wcss[i + 1] for i in range(len(wcss) - 1)]
print("\nDrop in WCSS when going from k to k+1:", [round(d) for d in drops])
# elbow = first k for which adding one more cluster reduces WCSS by less than
# 10% of the very first drop
k_best = next(k for k in range(1, len(drops) + 1) if drops[k - 1] < 0.1 * drops[0])
print("Adding more clusters after k =", k_best, "gives only a small drop, so the elbow is at k =", k_best)

# ---------------- K-Means with 5 clusters ----------------
km = KMeans(n_clusters=k_best, init="k-means++", n_init=10, random_state=42)
data["Cluster"] = km.fit_predict(X)
print("\nSilhouette score for k = 5:", round(silhouette_score(X, data["Cluster"]), 4))
print("\nCluster sizes:")
print(data["Cluster"].value_counts().sort_index().to_string())
print("\nCluster centres (income, spending score):")
print(pd.DataFrame(km.cluster_centers_.round(1), columns=["Annual Income (k$)", "Spending Score"]))
print("\nAverage values per cluster:")
print(data.groupby("Cluster")[["Age", "Annual Income (k$)", "Spending Score (1-100)"]].mean().round(1))

plt.figure(figsize=(8, 5))
colors = ["green", "orange", "brown", "dodgerblue", "red"]
for c in range(k_best):
    pts = data[data["Cluster"] == c]
    plt.scatter(pts["Annual Income (k$)"], pts["Spending Score (1-100)"], s=40, c=colors[c], label=f"Cluster
```

```
        {c}")
plt.scatter(km.cluster_centers_[:, 0], km.cluster_centers_[:, 1], s=200, c="black", marker="*",
    label="Centroids")
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("K-Means clusters (k = 5)")
plt.legend()
plt.savefig(os.path.join(OUT, "ML_KMeansClustering_pract12_clusters.png"), dpi=100)
plt.close()

# ---------------- K-Means on 3 features (Age, Income, Score) ----------------
X3 = data[["Age", "Annual Income (k$)", "Spending Score (1-100)"]].values
wcss3 = [KMeans(n_clusters=k, n_init=10, random_state=42).fit(X3).inertia_ for k in range(1, 11)]
print("\nWCSS with 3 features (k = 1..10):", [round(w) for w in wcss3])

# ---------------- Hierarchical clustering ----------------
Z = linkage(X, method="ward")
plt.figure(figsize=(10, 4))
dendrogram(Z, no_labels=True)
plt.title("Dendrogram (Ward linkage)")
plt.xlabel("Customers")
plt.ylabel("Euclidean distance")
plt.savefig(os.path.join(OUT, "ML_KMeansClustering_pract12_dendrogram.png"), dpi=100)
plt.close()

hc = AgglomerativeClustering(n_clusters=5, linkage="ward")
hc_labels = hc.fit_predict(X)
print("\nHierarchical clustering (5 clusters) sizes:", np.bincount(hc_labels).tolist())
print("Silhouette score (hierarchical):", round(silhouette_score(X, hc_labels), 4))
```

Output:

```
Shape: (200, 5)
   CustomerID  Gender  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Missing values: 0

Statistics:
       CustomerID     Age  Annual Income (k$)  Spending Score (1-100)
count      200.00  200.00              200.00                  200.00
mean       100.50   38.85               60.56                   50.20
std         57.88   13.97               26.26                   25.82
min          1.00   18.00               15.00                    1.00
25%         50.75   28.75               41.50                   34.75
50%        100.50   36.00               61.50                   50.00
75%        150.25   49.00               78.00                   73.00
max        200.00   70.00              137.00                   99.00

Correlation:
                        Age  Annual Income (k$)  Spending Score (1-100)
Age                   1.000              -0.012                  -0.327
Annual Income (k$)   -0.012               1.000                   0.010
Spending Score (1-100) -0.327               0.010                   1.000

WCSS for k = 1 to 10:
k = 1  WCSS = 269,981.3
k = 2  WCSS = 181,363.6
k = 3  WCSS = 106,348.4
k = 4  WCSS = 73,679.8
k = 5  WCSS = 44,448.5
k = 6  WCSS = 37,233.8
k = 7  WCSS = 30,241.3
k = 8  WCSS = 25,036.4
k = 9  WCSS = 21,916.8
k = 10 WCSS = 20,072.1

Drop in WCSS when going from k to k+1: [88618, 75015, 32669, 29231, 7215, 6992, 5205, 3120, 1845]
Adding more clusters after k = 5 gives only a small drop, so the elbow is at k = 5

Silhouette score for k = 5: 0.5539

Cluster sizes:
Cluster
0    81
1    39
```

```
2    22
3    35
4    23

Cluster centres (income, spending score):
   Annual Income (k$)  Spending Score
0                55.3            49.5
1                86.5            82.1
2                25.7            79.4
3                88.2            17.1
4                26.3            20.9

Average values per cluster:
          Age  Annual Income (k$)  Spending Score (1-100)
Cluster
0        42.7                55.3                    49.5
1        32.7                86.5                    82.1
2        25.3                25.7                    79.4
3        41.1                88.2                    17.1
4        45.2                26.3                    20.9

WCSS with 3 features (k = 1..10): [308813, 212840, 143343, 104366, 75351, 58300, 51133, 44640, 40810, 37128]

Hierarchical clustering (5 clusters) sizes: [32, 85, 39, 21, 23]
Silhouette score (hierarchical): 0.553
```

Figure: Elbow

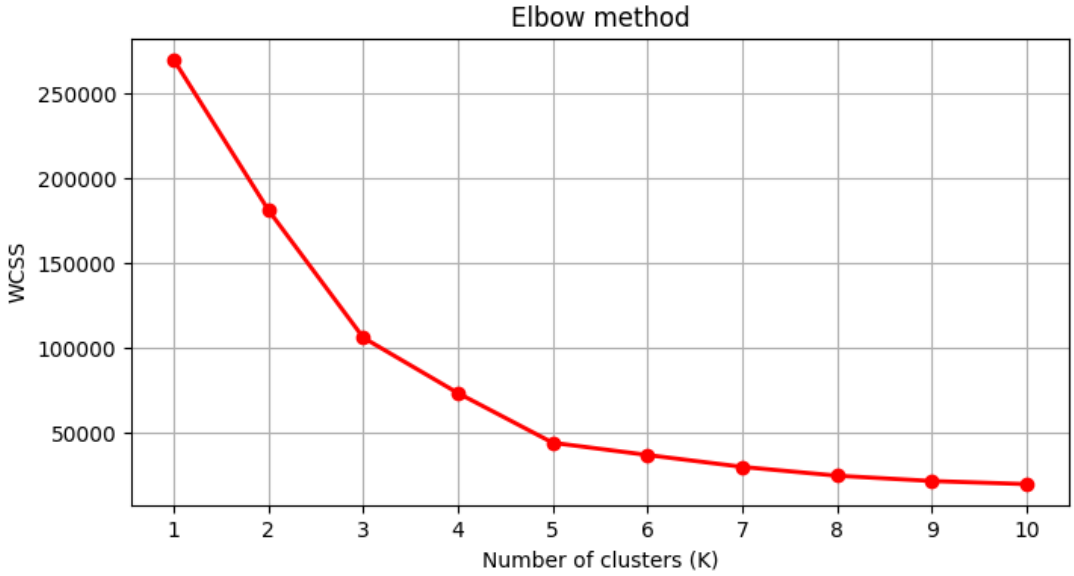


Figure: Clusters

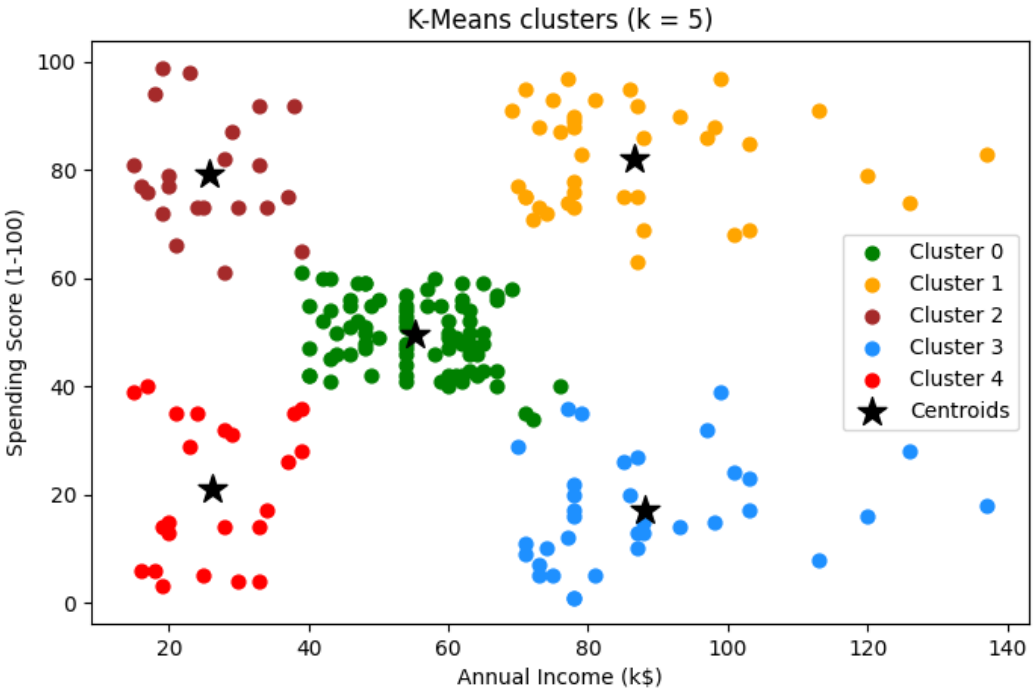
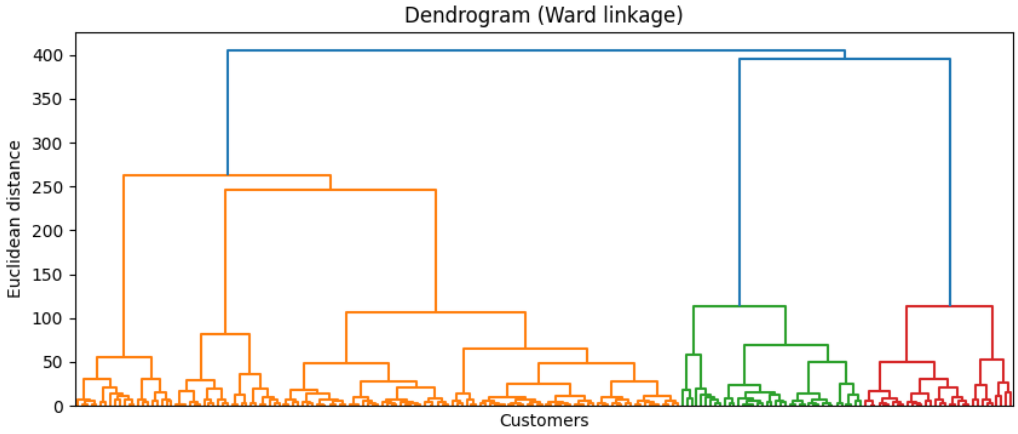


Figure: Dendrogram



Code:

```
# BT Assignment 15: MetaMask - study of Ether spent per transaction
# MetaMask itself is a browser extension (installed from the Chrome store), so it
# cannot be installed with Python. This program simulates two MetaMask accounts and
# shows how much Ether every transaction costs:
#   Transaction fee = Gas used x Gas price

WEI_PER_GWEI = 10 ** 9
WEI_PER_ETH = 10 ** 18
ETH_PRICE_USD = 2500          # assumed price, only used to show the fee in dollars


def gwei_to_eth(gwei):
    return gwei * WEI_PER_GWEI / WEI_PER_ETH


class Account:
    def __init__(self, name, balance_eth):
        self.name = name
        self.balance = balance_eth     # balance in ETH


class Network:
    """A very small test network that records every transaction."""

    def __init__(self):
        self.tx_list = []

    def send(self, sender, receiver, amount_eth, gas_used, gas_price_gwei, kind):
        fee = gas_used * gwei_to_eth(gas_price_gwei)      # fee in ETH
        total = amount_eth + fee                          # sender pays amount + fee
        if sender.balance < total:
            print(f"FAILED  : {sender.name} has {sender.balance:.6f} ETH, needs {total:.6f} ETH")
            return
        sender.balance -= total
        receiver.balance += amount_eth
        self.tx_list.append((len(self.tx_list) + 1, kind, amount_eth, gas_used, gas_price_gwei, fee))
        print(f"Tx {len(self.tx_list)} ({kind}): {sender.name} -> {receiver.name}, "
              f"{amount_eth} ETH, fee {fee:.6f} ETH")


print("Gas = unit of work, Gas price is paid in Gwei, 1 Gwei = 0.000000001 ETH")
print("Example: 21000 gas x 20 Gwei =", 21000 * gwei_to_eth(20), "ETH\n")

alice = Account("Account 1 (Alice)", 1.0)
bob = Account("Account 2 (Bob)", 0.5)
print(f"Start balances: Alice = {alice.balance} ETH, Bob = {bob.balance} ETH\n")

net = Network()
net.send(alice, bob, 0.10, 21000, 20, "ETH transfer")
net.send(alice, bob, 0.05, 21000, 50, "ETH transfer (fast, high gas price)")
net.send(bob, alice, 0.02, 21000, 10, "ETH transfer (slow, low gas price)")
net.send(alice, bob, 0.00, 65000, 25, "Smart contract call")
net.send(bob, alice, 0.01, 52000, 30, "Token transfer")
net.send(bob, alice, 5.00, 21000, 20, "ETH transfer (not enough balance)")

print("\nTransaction fee table")
print(f"{'No':<4}{'Type':<38}{'Gas used':<10}{'Gwei':<6}{'Fee (ETH)':<12}Fee (USD)")
for no, kind, amount, gas, price, fee in net.tx_list:
    print(f"{no:<4}{kind:<38}{gas:<10}{price:<6}{fee:<12.6f}${fee * ETH_PRICE_USD:.2f}")

total_fee = sum(t[5] for t in net.tx_list)
print(f"\nTotal fee spent on {len(net.tx_list)} transactions = {total_fee:.6f} ETH "
      f"(about ${total_fee * ETH_PRICE_USD:.2f} at an assumed ${ETH_PRICE_USD} per ETH)")
print(f"Average fee per transaction = {total_fee / len(net.tx_list):.6f} ETH")
print(f"\nFinal balances: Alice = {alice.balance:.6f} ETH, Bob = {bob.balance:.6f} ETH")

# EIP-1559 style fee (used by MetaMask today): gas used x (base fee + priority fee)
base_fee, priority_fee, gas_used = 15, 2, 21000
print(f"\nEIP-1559 example: {gas_used} x ({base_fee} + {priority_fee}) Gwei = "
      f"{gas_used * gwei_to_eth(base_fee + priority_fee):.6f} ETH")
```

Output:

```
Gas = unit of work, Gas price is paid in Gwei, 1 Gwei = 0.000000001 ETH
Example: 21000 gas x 20 Gwei = 0.00042 ETH

Start balances: Alice = 1.0 ETH, Bob = 0.5 ETH
Tx 1 (ETH transfer): Account 1 (Alice) -> Account 2 (Bob), 0.1 ETH, fee 0.000420 ETH
```

```
Tx 2 (ETH transfer (fast, high gas price)): Account 1 (Alice) -> Account 2 (Bob), 0.05 ETH, fee 0.001050 ETH
Tx 3 (ETH transfer (slow, low gas price)): Account 2 (Bob) -> Account 1 (Alice), 0.02 ETH, fee 0.000210 ETH
Tx 4 (Smart contract call): Account 1 (Alice) -> Account 2 (Bob), 0.0 ETH, fee 0.001625 ETH
Tx 5 (Token transfer): Account 2 (Bob) -> Account 1 (Alice), 0.01 ETH, fee 0.001560 ETH
FAILED  : Account 2 (Bob) has 0.618230 ETH, needs 5.000420 ETH

Transaction fee table
No  Type                                Gas used  Gwei  Fee (ETH)   Fee (USD)
1   ETH transfer                        21000     20    0.000420    $1.05
2   ETH transfer (fast, high gas price) 21000     50    0.001050    $2.62
3   ETH transfer (slow, low gas price)  21000     10    0.000210    $0.53
4   Smart contract call                 65000     25    0.001625    $4.06
5   Token transfer                      52000     30    0.001560    $3.90

Total fee spent on 5 transactions = 0.004865 ETH (about $12.16 at an assumed $2500 per ETH)
Average fee per transaction = 0.000973 ETH

Final balances: Alice = 0.876905 ETH, Bob = 0.618230 ETH

EIP-1559 example: 21000 x (15 + 2) Gwei = 0.000357 ETH
```

Code:

```
# BT Assignment 16: Create your own wallet for crypto transactions
# A real MetaMask wallet is created in the browser extension. This program shows
# the same ideas in a simple, educational way:
#   private key -> public key -> address, seed phrase, signing and a ledger.
# NOTE: simplified (uses SHA-256 and HMAC from Python's standard library). Real
# Ethereum wallets use secp256k1 keys, Keccak-256 and ECDSA signatures.

import hashlib
import hmac
import secrets
import time

WORDS = ["apple", "river", "stone", "cloud", "tiger", "lemon", "orbit", "sword",
         "piano", "ocean", "maple", "eagle", "flame", "coral", "ember", "frost",
         "grape", "honey", "ivory", "jungle", "karma", "lunar", "mango", "noble"]


class Wallet:
    def __init__(self, name):
        self.name = name
        # 12-word secret recovery phrase (like MetaMask shows when you create a wallet)
        self.seed_phrase = [secrets.choice(WORDS) for _ in range(12)]
        # private key derived from the phrase - never share it
        self.private_key = hashlib.sha256(" ".join(self.seed_phrase).encode()).hexdigest()
        # public key derived from the private key (one-way)
        self.public_key = hashlib.sha256(self.private_key.encode()).hexdigest()
        # address = "0x" + last 20 bytes (40 hex characters) of the hash of the public key
        self.address = "0x" + hashlib.sha256(self.public_key.encode()).hexdigest()[-40:]
        self.balance = 0.0

    def sign(self, message):
        """Digital signature proves the owner of the private key approved the transaction."""
        return hmac.new(bytes.fromhex(self.private_key), message.encode(), hashlib.sha256).hexdigest()


class Ledger:
    """Public ledger of all transactions (a very small blockchain-like list)."""

    def __init__(self):
        self.transactions = []

    def transfer(self, sender, receiver, amount, fee=0.0005):
        msg = f"{sender.address}->{receiver.address}:{amount}"
        if sender.balance < amount + fee:
            print(f"Transaction rejected: {sender.name} has insufficient balance")
            return None
        sender.balance -= amount + fee
        receiver.balance += amount
        tx_hash = "0x" + hashlib.sha256((msg + str(time.time_ns())).encode()).hexdigest()
        self.transactions.append({"hash": tx_hash, "from": sender.address, "to": receiver.address,
                                  "amount": amount, "fee": fee, "signature": sender.sign(msg)})
        print(f"Transaction successful: {sender.name} sent {amount} ETH to {receiver.name} (fee {fee} ETH)")
        return tx_hash


# ---------------- Create two wallets ----------------
alice = Wallet("Alice")
bob = Wallet("Bob")

for w in (alice, bob):
    print(f"--- Wallet of {w.name} ---")
    print("Secret recovery phrase :", " ".join(w.seed_phrase))
    print("Private key (secret)   :", w.private_key[:16] + "... (hidden)")
    print("Public key             :", w.public_key[:16] + "...")
    print("Address                :", w.address)
    print()

# ---------------- Fund the wallet (like a test-network faucet) ----------------
alice.balance = 2.0
print("Faucet sent 2.0 test ETH to Alice\n")

# ---------------- Transactions ----------------
ledger = Ledger()
ledger.transfer(alice, bob, 0.5)
ledger.transfer(bob, alice, 0.1)
ledger.transfer(bob, alice, 10)          # fails: not enough balance

print("\nBalances")
```

```
print(f"Alice: {alice.balance:.4f} ETH")
print(f"Bob  : {bob.balance:.4f} ETH")

print("\nLedger")
for i, tx in enumerate(ledger.transactions, start=1):
    print(f"Tx {i}: hash {tx['hash'][:18]}... from {tx['from'][:10]}... to {tx['to'][:10]}... "
          f"amount {tx['amount']} ETH, fee {tx['fee']} ETH")
    print(f"      signature {tx['signature'][:24]}...")

# same phrase always gives the same wallet (this is how wallet recovery works)
restored_key = hashlib.sha256(" ".join(alice.seed_phrase).encode()).hexdigest()
print("\nRestoring Alice's wallet from the recovery phrase gives the same key:",
      restored_key == alice.private_key)
```

Output:

```
--- Wallet of Alice ---
Secret recovery phrase : lemon ember frost flame grape frost noble river ocean cloud coral tiger
Private key (secret)   : c520c42e9e62af28... (hidden)
Public key             : dd3d2d7dd0ad6efc...
Address                : 0xf14397dced93601c579a5d23af70e12519e67880

--- Wallet of Bob ---
Secret recovery phrase : stone tiger stone apple flame piano lemon eagle frost coral honey maple
Private key (secret)   : 89da04b6ce79b701... (hidden)
Public key             : 88f899b0122302b5...
Address                : 0x9b475e5bbf9be724bba36322f6df403614ee472c

Faucet sent 2.0 test ETH to Alice

Transaction successful: Alice sent 0.5 ETH to Bob (fee 0.0005 ETH)
Transaction successful: Bob sent 0.1 ETH to Alice (fee 0.0005 ETH)
Transaction rejected: Bob has insufficient balance

Balances
Alice: 1.5995 ETH
Bob  : 0.3995 ETH

Ledger
Tx 1: hash 0x58ec6b36648c3b22... from 0xf14397dc... to 0x9b475e5b... amount 0.5 ETH, fee 0.0005 ETH
      signature 6ae6a558b749b099e3104498...
Tx 2: hash 0xc77e3bb69a4c29fc... from 0x9b475e5b... to 0xf14397dc... amount 0.1 ETH, fee 0.0005 ETH
      signature 68a90094ce0272010977f13d...

Restoring Alice's wallet from the recovery phrase gives the same key: True
```

Code:

```
# BT Assignment 17: Smart contract for a bank account - deposit, withdraw, show balance
# The practical writes this contract in Solidity and deploys it on a test network
# (Remix + MetaMask). Here the SAME logic is written in Python so it can be run and
# tested without a blockchain:
#   mapping(address => uint256) public balanceOf;   ->  self.balance_of (dictionary)
#   require(condition)                              ->  require() raises an error (revert)
#   msg.sender                                      ->  the "sender" argument


class RevertError(Exception):
    """Same as a Solidity revert: the transaction is cancelled, nothing is changed."""


def require(condition, message):
    if not condition:
        raise RevertError(message)


class Bank:
    def __init__(self):
        self.balance_of = {}            # balance of every depositor, default 0
        self.contract_balance = 0       # total ether held by the contract
        self.events = []                # log of Deposit / Withdraw events

    def deposit(self, sender, amount):
        require(amount > 0, "Deposit amount must be greater than 0")
        self.balance_of[sender] = self.balance_of.get(sender, 0) + amount
        self.contract_balance += amount
        self.events.append(("Deposit", sender, amount))

    def withdraw(self, sender, amount):
        require(amount > 0, "Withdraw amount must be greater than 0")
        require(amount <= self.balance_of.get(sender, 0), "Insufficient balance")
        # Checks-Effects-Interactions: update the balance BEFORE sending the money
        self.balance_of[sender] -= amount
        self.contract_balance -= amount
        self.events.append(("Withdraw", sender, amount))
        return amount                   # "transfer" of ether back to the sender

    def get_balance(self, sender):
        return self.balance_of.get(sender, 0)


def attempt(description, function, *args):
    """Run a transaction and show whether it succeeded or reverted."""
    try:
        result = function(*args)
        print(f"OK       {description}")
        return result
    except RevertError as error:
        print(f"REVERTED {description} -> {error}")


alice = "0xAlice0000000000000000000000000000000001"
bob = "0xBob00000000000000000000000000000000002"

bank = Bank()
print("Contract deployed. Balance of Alice:", bank.get_balance(alice), "ETH\n")

attempt("Alice deposits 5 ETH", bank.deposit, alice, 5)
attempt("Bob deposits 3 ETH", bank.deposit, bob, 3)
print("   Alice balance =", bank.get_balance(alice), " Bob balance =", bank.get_balance(bob),
      " Contract holds =", bank.contract_balance)

attempt("Alice withdraws 2 ETH", bank.withdraw, alice, 2)
print("   Alice balance =", bank.get_balance(alice))

attempt("Alice withdraws 10 ETH (more than she has)", bank.withdraw, alice, 10)
attempt("Bob withdraws 3 ETH (everything)", bank.withdraw, bob, 3)
attempt("Bob withdraws 1 ETH (nothing left)", bank.withdraw, bob, 1)
attempt("Alice deposits 0 ETH", bank.deposit, alice, 0)

print("\nFinal balances")
print("Alice :", bank.get_balance(alice), "ETH")
print("Bob   :", bank.get_balance(bob), "ETH")
print("Contract holds", bank.contract_balance, "ETH (= sum of all balances:",
      sum(bank.balance_of.values()), ")")
print("\nEvent log")
```

```
for i, (name, who, amount) in enumerate(bank.events, start=1):
    print(f"{i}. {name:<9} {who[:10]}...  {amount} ETH")
```

Output:

```
Contract deployed. Balance of Alice: 0 ETH

OK       Alice deposits 5 ETH
OK       Bob deposits 3 ETH
   Alice balance = 5  Bob balance = 3  Contract holds = 8
OK       Alice withdraws 2 ETH
   Alice balance = 3
REVERTED Alice withdraws 10 ETH (more than she has) -> Insufficient balance
OK       Bob withdraws 3 ETH (everything)
REVERTED Bob withdraws 1 ETH (nothing left) -> Insufficient balance
REVERTED Alice deposits 0 ETH -> Deposit amount must be greater than 0

Final balances
Alice : 3 ETH
Bob   : 0 ETH
Contract holds 3 ETH (= sum of all balances: 3 )

Event log
1. Deposit   0xAlice000...  5 ETH
2. Deposit   0xBob00000...  3 ETH
3. Withdraw  0xAlice000...  2 ETH
4. Withdraw  0xBob00000...  3 ETH
```

Code:

```
# BT Assignment 18: Student data contract - structure, array and fallback function,
# with transaction fee and gas values.
# The practical deploys a Solidity contract with Remix on an Ethereum test network.
# Here the contract is modelled in Python:
#   struct Student {...}        ->  dataclass Student
#   Student[] public students   ->  list self.students
#   addStudent / getStudent     ->  methods with the same names
#   fallback() external payable ->  fallback() method (called for unknown calls / plain ETH)
# Gas numbers are ESTIMATES based on typical Ethereum costs (the real values are
# read from Remix / MetaMask after deployment).

from dataclasses import dataclass

TX_BASE_GAS = 21000          # every transaction costs at least this much
STORAGE_WORD_GAS = 22100     # writing one new 32-byte storage slot
DEPLOY_GAS = 450000          # assumed cost of deploying this small contract
GAS_PRICE_GWEI = 20          # assumed gas price
GWEI = 10 ** 9
ETH = 10 ** 18


@dataclass
class Student:
    id: int
    name: str
    age: int
    course: str


class StudentData:
    def __init__(self):
        self.students = []           # dynamic array of Student structures
        self.balance = 0             # ether received through the fallback function

    def add_student(self, id, name, age, course):
        self.students.append(Student(id, name, age, course))
        # id, age, name and course use 4 storage slots (short strings fit in one slot)
        return TX_BASE_GAS + 4 * STORAGE_WORD_GAS

    def get_student(self, index):
        return self.students[index]          # a read-only call costs no gas

    def get_count(self):
        return len(self.students)

    def fallback(self, value=0):
        """Runs when a call does not match any function or when plain ETH is sent."""
        self.balance += value
        return TX_BASE_GAS + 2300


def fee_in_eth(gas_used):
    return gas_used * GAS_PRICE_GWEI * GWEI / ETH


def show_tx(label, gas_used):
    print(f"{label:<40}{gas_used:<12}{GAS_PRICE_GWEI:<12}{fee_in_eth(gas_used):.6f}")


print(f"Gas price assumed: {GAS_PRICE_GWEI} Gwei     Transaction fee = Gas used x Gas price\n")
print(f"{'Transaction':<40}{'Gas used':<12}{'Gwei':<12}Fee (ETH)")

contract = StudentData()
show_tx("Deploy StudentData contract", DEPLOY_GAS)

records = [(1, "Aarav Sharma", 20, "Computer Engineering"),
           (2, "Diya Patil", 21, "Information Technology"),
           (3, "Rohan Mehta", 19, "Electronics")]
total_gas = DEPLOY_GAS
for r in records:
    gas = contract.add_student(*r)
    total_gas += gas
    show_tx(f"addStudent({r[0]}, '{r[1]}', ...)", gas)

gas = contract.fallback(value=1)           # someone sends ETH to the contract
total_gas += gas
show_tx("Fallback (plain ETH sent)", gas)
print(f"\nTotal gas used  = {total_gas}")
```

```
print(f"Total fee       = {fee_in_eth(total_gas):.6f} ETH")
add_gas = TX_BASE_GAS + 4 * STORAGE_WORD_GAS
print(f"Deployment costs {DEPLOY_GAS / add_gas:.1f} times as much gas as adding one student.")

print("\nStudents stored in the contract (getStudent / public array getter):")
print(f"Total students = {contract.get_count()}")
for i in range(contract.get_count()):
    s = contract.get_student(i)
    print(f"students[{i}] -> id={s.id}, name={s.name}, age={s.age}, course={s.course}")

print("\nEther held by the contract (from the fallback function):", contract.balance, "ETH")
```

Output:

```
Gas price assumed: 20 Gwei     Transaction fee = Gas used x Gas price

Transaction                             Gas used    Gwei        Fee (ETH)
Deploy StudentData contract             450000      20          0.009000
addStudent(1, 'Aarav Sharma', ...)      109400      20          0.002188
addStudent(2, 'Diya Patil', ...)        109400      20          0.002188
addStudent(3, 'Rohan Mehta', ...)       109400      20          0.002188
Fallback (plain ETH sent)               23300       20          0.000466

Total gas used  = 801500
Total fee       = 0.016030 ETH
Deployment costs 4.1 times as much gas as adding one student.

Students stored in the contract (getStudent / public array getter):
Total students = 3
students[0] -> id=1, name=Aarav Sharma, age=20, course=Computer Engineering
students[1] -> id=2, name=Diya Patil, age=21, course=Information Technology
students[2] -> id=3, name=Rohan Mehta, age=19, course=Electronics

Ether held by the contract (from the fallback function): 1 ETH
```

Code:

```
# BT Assignment 19: Survey report on types of blockchains and real-time use cases
# The survey data is stored in Python dictionaries and printed as a report.

import os
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

BASE = os.path.dirname(os.path.abspath(__file__))
OUT = os.path.join(BASE, "outputs")
os.makedirs(OUT, exist_ok=True)

survey = {
    "Public": {
        "who can join": "Anyone with a computer and internet",
        "controlled by": "Nobody (fully decentralized)",
        "consensus": "Proof of Work / Proof of Stake",
        "advantages": ["Trustable", "Secure (large network)", "Anonymous", "Decentralized"],
        "disadvantages": ["Slow transactions", "High energy use (PoW)", "Hard for governments to accept"],
        "examples": ["Bitcoin", "Ethereum"],
        "use cases": ["Cryptocurrency payments", "Smart contracts / DApps", "Replacing traditional finance"],
        # ratings from 1 (low) to 5 (high) used for the comparison chart
        "scores": {"Decentralization": 5, "Speed": 1, "Privacy": 2, "Security": 5, "Scalability": 2},
    },
    "Private": {
        "who can join": "Only selected, authorized users of one organization",
        "controlled by": "A single organization",
        "consensus": "Permissioned (e.g. PBFT / Raft)",
        "advantages": ["Fast transactions", "Scalable", "High privacy", "Balanced access"],
        "disadvantages": ["Fewer nodes, easier to manipulate", "Centralized, trust issue", "System at risk if
            nodes go offline"],
        "examples": ["Hyperledger", "Corda"],
        "use cases": ["Internal auditing", "Voting", "Asset management"],
        "scores": {"Decentralization": 1, "Speed": 5, "Privacy": 5, "Security": 3, "Scalability": 4},
    },
    "Hybrid": {
        "who can join": "Part is public, part is private (permission based)",
        "controlled by": "One organization, but it cannot alter transactions",
        "consensus": "Mixed (permissioned + permissionless)",
        "advantages": ["Cannot be easily hacked", "Cheap transactions", "Customizable", "Chooses what is
            public"],
        "disadvantages": ["Difficult to implement", "Less transparency", "Few incentives for participants"],
        "examples": ["Ripple network", "XRP token"],
        "use cases": ["Healthcare", "Government", "Real estate", "Financial companies"],
        "scores": {"Decentralization": 3, "Speed": 4, "Privacy": 4, "Security": 4, "Scalability": 4},
    },
    "Consortium": {
        "who can join": "Members of a group of organizations",
        "controlled by": "More than one organization (federated)",
        "consensus": "Permissioned (e.g. Tendermint BFT)",
        "advantages": ["Fast verification", "Shared authority, more secure", "Private to outsiders",
            "Flexible"],
        "disadvantages": ["All members must approve protocol", "Can be hacked if an organization is corrupt",
            "Few nodes, higher vulnerability"],
        "examples": ["Tendermint", "Multichain"],
        "use cases": ["Banks and payment processors", "Food / supply-chain tracking", "Business-to-business data
             sharing"],
        "scores": {"Decentralization": 3, "Speed": 4, "Privacy": 4, "Security": 3, "Scalability": 4},
    },
}

print("SURVEY REPORT: TYPES OF BLOCKCHAINS AND THEIR REAL-TIME USE CASES")
print("=" * 66)

for number, (name, info) in enumerate(survey.items(), start=1):
    print(f"\n{number}. {name.upper()} BLOCKCHAIN")
    print("-" * 40)
    print("Who can join  :", info["who can join"])
    print("Controlled by :", info["controlled by"])
    print("Consensus     :", info["consensus"])
    print("Advantages    :", "; ".join(info["advantages"]))
    print("Disadvantages :", "; ".join(info["disadvantages"]))
    print("Examples      :", ", ".join(info["examples"]))
    print("Use cases     :", "; ".join(info["use cases"]))

# ---------------- Comparison table ----------------
criteria = list(next(iter(survey.values()))["scores"])
print("\n\nCOMPARISON (1 = low, 5 = high)")
```

```
print(f"{'Type':<12}" + "".join(f"{c:<18}" for c in criteria) + "Total")
for name, info in survey.items():
    scores = info["scores"]
    print(f"{name:<12}" + "".join(f"{scores[c]:<18}" for c in criteria) + str(sum(scores.values())))

# ---------------- Which blockchain for which need? ----------------
print("\n\nWHICH BLOCKCHAIN SHOULD BE USED?")
needs = {
    "Open cryptocurrency anyone can use": "Public",
    "Company's internal audit trail": "Private",
    "Hospital sharing some data publicly, some privately": "Hybrid",
    "Banks and suppliers sharing one ledger": "Consortium",
}
for need, answer in needs.items():
    print(f"{need:<55} -> {answer}")

# ---------------- Chart ----------------
x = range(len(criteria))
width = 0.2
plt.figure(figsize=(10, 4.5))
for k, (name, info) in enumerate(survey.items()):
    plt.bar([i + k * width for i in x], [info["scores"][c] for c in criteria], width, label=name)
plt.xticks([i + 1.5 * width for i in x], criteria)
plt.ylabel("Rating (1-5)")
plt.title("Comparison of blockchain types")
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(OUT, "BT_BlockchainTypesSurvey_pract19_comparison.png"), dpi=100)
plt.close()

print("\nConclusion: public chains favour openness, private chains favour speed and privacy,")
print("hybrid and consortium chains balance both for business use.")
```

Output:

```
SURVEY REPORT: TYPES OF BLOCKCHAINS AND THEIR REAL-TIME USE CASES
================================================================

1. PUBLIC BLOCKCHAIN
----------------------------------------
Who can join  : Anyone with a computer and internet
Controlled by : Nobody (fully decentralized)
Consensus     : Proof of Work / Proof of Stake
Advantages    : Trustable; Secure (large network); Anonymous; Decentralized
Disadvantages : Slow transactions; High energy use (PoW); Hard for governments to accept
Examples      : Bitcoin, Ethereum
Use cases     : Cryptocurrency payments; Smart contracts / DApps; Replacing traditional finance

2. PRIVATE BLOCKCHAIN
----------------------------------------
Who can join  : Only selected, authorized users of one organization
Controlled by : A single organization
Consensus     : Permissioned (e.g. PBFT / Raft)
Advantages    : Fast transactions; Scalable; High privacy; Balanced access
Disadvantages : Fewer nodes, easier to manipulate; Centralized, trust issue; System at risk if nodes go offline
Examples      : Hyperledger, Corda
Use cases     : Internal auditing; Voting; Asset management

3. HYBRID BLOCKCHAIN
----------------------------------------
Who can join  : Part is public, part is private (permission based)
Controlled by : One organization, but it cannot alter transactions
Consensus     : Mixed (permissioned + permissionless)
Advantages    : Cannot be easily hacked; Cheap transactions; Customizable; Chooses what is public
Disadvantages : Difficult to implement; Less transparency; Few incentives for participants
Examples      : Ripple network, XRP token
Use cases     : Healthcare; Government; Real estate; Financial companies

4. CONSORTIUM BLOCKCHAIN
----------------------------------------
Who can join  : Members of a group of organizations
Controlled by : More than one organization (federated)
Consensus     : Permissioned (e.g. Tendermint BFT)
Advantages    : Fast verification; Shared authority, more secure; Private to outsiders; Flexible
Disadvantages : All members must approve protocol; Can be hacked if an organization is corrupt; Few nodes,
    higher vulnerability
Examples      : Tendermint, Multichain
Use cases     : Banks and payment processors; Food / supply-chain tracking; Business-to-business data sharing
COMPARISON (1 = low, 5 = high)
Type        Decentralization  Speed             Privacy           Security          Scalability       Total
```

```
Public      5               1               2               5               2               15
Private     1               5               5               3               4               18
Hybrid      3               4               4               4               4               19
Consortium  3               4               4               3               4               18


WHICH BLOCKCHAIN SHOULD BE USED?
Open cryptocurrency anyone can use                      -> Public
Company's internal audit trail                          -> Private
Hospital sharing some data publicly, some privately     -> Hybrid
Banks and suppliers sharing one ledger                  -> Consortium

Conclusion: public chains favour openness, private chains favour speed and privacy,
hybrid and consortium chains balance both for business use.
```

Figure: Comparison

